A Zero-Knowledge Decentralized Identity Protocol for Privacy-Preserving Mustahik Verification in Blockchain-Based Zakat Management Systems

Muhammad Zidan Fatonie
School of Computer Science
BINUS University
Jakarta, Indonesia
mzidanfatonie@gmail.com

Alexander Agung Santoso Gunawan
School of Computer Science
BINUS University
Jakarta, Indonesia
aagung@binus.ac.id

***Abstract*—Zakat, one of the five pillars of Islam, constitutes a mandatory system of wealth redistribution with profound social welfare implications. Despite increasing digitization of zakat management, contemporary systems remain burdened by critical deficiencies: opacity in fund disbursement, centralized control susceptible to corruption, and, most critically, the exposure of *mustahik* (zakat recipients) personal and socioeconomic data during eligibility verification. This paper proposes a novel Zero-Knowledge Decentralized Identity (ZK-DID) protocol specifically engineered for privacy-preserving mustahik verification within blockchain-based zakat management systems. The proposed architecture integrates zero-knowledge proof systems—specifically zk-SNARKs (Zero-Knowledge Succinct Non-Interactive Arguments of Knowledge)—with W3C Decentralized Identifier standards to enable cryptographic verification of a recipient's eligibility without disclosing underlying personal attributes or financial records. The current protocol instantiation targets the *fakir* (destitute) and *miskin* (poor) *asnaf* categories, which admit quantifiable income and asset predicates; the architecture is designed to be extensible to the remaining six categories. The protocol is designed to satisfy simultaneous requirements of privacy, verifiability, scalability, and Sharia compliance. A prototype implementation is deployed on the Base PoS network using Circom, SnarkJS, and Solidity. Experimental evaluation focuses on proof generation overhead, on-chain verification gas costs, and resistance to identity-linkage attacks. This work contributes a domain-specific identity layer addressing both the technical and ethical imperatives of privacy in Islamic social finance infrastructure.**

***Index Terms*—Zero-Knowledge Proofs, zk-SNARKs, Decentralized Identity, Blockchain, Zakat Management, Mustahik Verification, Privacy-Preserving Protocols, Islamic FinTech, Self-Sovereign Identity**

I. Introduction

A. Background and Motivation

Zakat management in the contemporary landscape stands at the intersection of centuries-old Islamic jurisprudence and modern digital infrastructure demands. Globally, zakat collection is estimated to reach between USD 200 billion and USD 1 trillion annually, yet a significant portion fails to reach legitimate *mustahik* due to systemic inefficiencies, misappropriation, and institutional distrust [1]. The opacity inherent in centralized zakat institutions has eroded public confidence, while manual verification processes for mustahik eligibility remain vulnerable to fraud and administrative bottlenecks.

The advent of distributed ledger technology has prompted research into blockchain-based zakat systems offering immutability, transparency, and automated disbursement via smart contracts. Pioneering works by Mustafa et al. [2] and Khatiman et al. [3] demonstrated that Ethereum-based zakat platforms can significantly reduce intermediary overhead and improve auditability. However, these systems introduce a critical paradox: by placing transaction records on a public ledger, they inadvertently expose the socioeconomic status and personal circumstances of mustahik to public scrutiny. The stigma associated with receiving charity in Muslim-majority societies means that such exposure can have serious psychosocial and economic consequences for recipients [2].

Zero-Knowledge Proofs (ZKPs), formalized by Goldwasser, Micali, and Rackoff [5], offer a compelling cryptographic resolution. A ZKP allows a prover to convince a verifier that a statement is true without revealing any information beyond the truth of that statement. In the context of mustahik verification, a ZKP-based system allows a recipient to prove eligibility criteria—such as income below a defined threshold or asset holdings within permissible bounds—without disclosing actual attribute values. Recent advances in succinct non-interactive ZKP constructions, particularly Groth16 [6] and PLONK [7], have made on-chain proof verification computationally feasible.

Decentralized Identity (DID) standards, as defined by the W3C DID Core specification [8], enable self-sovereign identity management in which individuals control their own credentials without dependence on a central authority. Combining ZKPs with DID frameworks creates a powerful identity layer capable of selective disclosure. This paradigm aligns with the *maqasid al-shariah* (objectives of Islamic law), specifically the protection of *nafs* (life and dignity) and *mal* (wealth) of individuals.

Despite the theoretical promise of this convergence, no prior work has proposed a domain-specific ZK-DID protocol tailored to the unique requirements of zakat management—

including Sharia-compliant eligibility criteria, multi-criterion *nisab* assessment, and integration with existing *amil* (zakat institution) infrastructure. This paper addresses that gap.

B. Research Objectives

The primary objective of this research is to design, implement, and evaluate a ZK-DID protocol for privacy-preserving mustahik verification in blockchain-based zakat management systems. The specific objectives are:

1) To formalize mustahik eligibility requirements as cryptographic predicates expressible within a zk-SNARK circuit framework.
2) To design a ZK-DID protocol architecture integrating W3C DID standards with zk-SNARK-based credential verification.
3) To implement a prototype system on an EVM-compatible blockchain with on-chain proof verification and DID registry management.
4) To evaluate the protocol with respect to computational overhead, on-chain gas costs, and privacy guarantees against identity-linkage attacks.
5) To assess practical deployment feasibility within existing *amil* operational frameworks, including Sharia compliance considerations.

C. Research Questions

This study addresses the following research questions:

RQ1: How can zero-knowledge proof systems formally encode Sharia-compliant mustahik eligibility criteria as verifiable predicates without exposing personal or financial attributes?

RQ2: What architectural integration is required to couple W3C DID standards with zk-SNARK-based credential verification in a blockchain-native zakat management context?

RQ3: What are the practical computational and economic trade-offs of the proposed ZK-DID protocol at scale?

RQ4: To what extent does the proposed protocol satisfy formal privacy properties and operational requirements of *amil* institutions?

D. Contributions

This paper makes the following contributions:

- **Domain-specific ZKP circuit design**: The first formalization of Islamic zakat eligibility predicates (*nisab*, *hawl*) for the *fakir* and *miskin asnaf* categories as R1CS arithmetic circuits for zk-SNARK proving, with an extensible architecture for the remaining six categories.
- **ZK-DID protocol for Islamic social finance**: A complete five-actor protocol architecture integrating W3C DID, Verifiable Credentials, and Groth16 proofs for mustahik verification.
- **Open-source prototype**: A working implementation on Base PoS using Circom, SnarkJS, and Solidity, with a Progressive Web Application claimant wallet.
- **Empirical evaluation**: Performance benchmarks, security analysis, and usability assessment grounded in the Technology Acceptance Model (TAM).

II. Background and Related Work

A. Blockchain-Based Zakat Management Systems

The intersection of blockchain technology and Islamic social finance has attracted increasing scholarly attention since 2017. Mustafa et al. [2] proposed a foundational Ethereum-based zakat distribution framework employing ERC-20 tokens to represent zakat funds, demonstrating that smart contract automation could eliminate intermediary delays and provide an immutable audit trail, reducing disbursement processing time by approximately 60% in a simulated institutional environment. However, the system inherited all transparency properties of the underlying public blockchain, inadvertently exposing recipient addresses to potential de-anonymization through transaction graph analysis.

Khatiman et al. [3] extended this work by incorporating a multi-signature governance mechanism for high-value disbursements using Solidity smart contracts on Ethereum. While this improved accountability, the architecture remained centered on pseudonymous addresses rather than privacy-preserving identity constructs. Subsequent work by Shalaby et al. [4] explored Hyperledger Fabric as an alternative model, arguing that permissioned ledgers provide de facto privacy through access control. This approach, however, introduces centralization trade-offs and undermines trustless verification.

The literature reveals a persistent tension: transparency, necessary for institutional accountability, conflicts directly with the privacy needs of mustahik recipients. This paper positions ZKPs as the mechanism through which both properties can be simultaneously satisfied.

B. Zero-Knowledge Proof Systems

The theoretical foundations of zero-knowledge proofs were established by Goldwasser, Micali, and Rackoff [5], defining three fundamental properties: *completeness* (an honest prover always convinces an honest verifier), *soundness* (a dishonest prover cannot convince a verifier of a false statement except with negligible probability), and *zero-knowledge* (the proof reveals nothing beyond the truth of the statement).

Groth [6] introduced the Groth16 construction, a pairing-based zk-SNARK requiring a circuit-specific trusted setup, which remains the most widely deployed proof system due to its highly optimized verification time ($\mathcal{O}(1)$ pairing operations) and proof size of approximately 192 bytes. The Universal PLONK construction by Gabizon, Williamson, and Ciobotaru [7] introduced a universal and updateable structured reference string (SRS), permitting a single trusted setup ceremony to be reused across multiple circuits. For the purposes of this work, Groth16 is selected as the primary proof system due to its maturity and minimal on-chain verification cost.

C. Decentralized Identity and Self-Sovereign Identity

The W3C DID Core specification [8] defines a global standard for decentralized, cryptographically verifiable identifiers independent of any central registry. A DID resolves to a DID Document containing cryptographic material (public keys) and service endpoints. The W3C Verifiable Credentials Data Model

[9] provides a standard envelope for cryptographically signed attribute assertions issued by credential issuers.

Self-Sovereign Identity (SSI) architectures have been extensively examined in the literature. Muhle et al. [10] provide a comprehensive survey identifying the three-party trust triangle (issuer-holder-verifier) as the foundational model. The BBS+ signature scheme [11] has emerged as a preferred primitive for ZK-compatible credential systems due to its support for efficient proofs of knowledge of signed messages.

D. Privacy-Preserving Credential Verification

The combination of ZKPs with verifiable credentials has been explored in several recent works. Bichsel et al. [12] introduced the Privacy-ABC framework, later formalized in the ABC4Trust project [13]. Pauwels [14] proposed zkKYC, a zero-knowledge Know-Your-Customer framework for regulated financial environments, demonstrating that ZK-based compliance verification is achievable within existing AML/CFT frameworks. Their work is directly analogous to the zakat eligibility use case, though without domain-specific Sharia requirements.

Range proofs—ZKPs demonstrating a committed value lies within a specified interval—are central to income and asset verification. Bulletproofs [15] provide logarithmically sized range proofs without a trusted setup. However, their linear verification cost presents scalability challenges for high-throughput zakat disbursement cycles, motivating the use of Groth16-based range subcircuits in this work.

E. Research Gap

Despite substantial literature addressing blockchain-based zakat management, ZKP systems, DID standards, and privacy-preserving credential verification as independent domains, a critical convergence gap remains. Three interconnected gaps are identified:

1) **Privacy in zakat systems**: Existing blockchain-based zakat platforms [2]–[4] have not integrated privacy-preserving identity mechanisms beyond pseudonymous addressing.
2) **Domain-specific ZKP circuits**: Existing ZKP credential frameworks [13], [14] are designed for generalized financial compliance and do not encode Islamic jurisprudential eligibility criteria—*nisab* thresholds, *hawl* assessment periods, and domain-specific *asnaf* predicates; the current instantiation targets the *fakir* and *miskin* categories as the primary quantifiable predicates.
3) **ZK-DID for Islamic finance**: The integration of W3C DID standards with ZKP-based credential verification has not been explored within the Islamic social finance domain.

This paper directly addresses all three gaps.

III. Proposed ZK-DID Protocol

A. System Overview and Threat Model

The ZK-DID protocol operates within a five-actor trust model: the *Government Data Authority* (credential issuer), the *Mustahik Claimant* (credential holder and ZKP prover), the *Amil Institution* (verifier and on-chain transaction initiator), the *Blockchain Network* (verification and disbursement execution), and the *DID Registry* (on-chain identifier resolution).

The threat model assumes an honest-but-curious verifier (the amil institution may attempt to infer private attributes from submitted proofs), a potentially malicious prover (a non-eligible claimant may attempt to fabricate a proof), and a public blockchain network subject to transaction graph analysis. The protocol provides:

- **Zero-knowledge**: The amil institution learns nothing about a claimant's private attributes beyond eligibility satisfaction.
- **Soundness**: A non-eligible claimant cannot generate an accepted proof except with negligible probability under the q-PKE assumption.
- **Double-claiming prevention**: A nullifier hash mechanism prevents the same mustahik from claiming zakat more than once per cycle.
- **Sybil resistance**: DID-based identity binding prevents a single claimant from registering multiple pseudonymous identities.

B. Formal Specification of Eligibility Predicates

The initial phase formalizes mustahik eligibility criteria as verifiable computational predicates. The research focuses on the *fakir* (destitute) and *miskin* (poor) categories, which represent the largest recipient populations under classical Hanafi jurisprudence.

Definition 1 (Eligibility Predicate). Let $\mathbf{A} = \{a_1, a_2, \ldots, a_n\}$ be the set of private attribute values held by a mustahik claimant (e.g., monthly income a_1, total non-exempt assets a_2, number of dependents a_3). Let $\mathbf{T} = \{t_1, t_2, \ldots, t_n\}$ be the publicly known threshold values for the current zakat cycle (e.g., nisab value in IDR t_1, income ceiling t_2). The eligibility predicate P is defined as a Boolean circuit C such that:

$$C(\mathbf{A}, \mathbf{T}) = 1 \iff \bigwedge_{i=1}^{k} \phi_i(a_i, t_i) \tag{1}$$

where each ϕ_i is a range or membership predicate (e.g., $a_1 \leq t_1$ for income below nisab threshold).

Definition 2 (ZK Statement). The zero-knowledge statement to be proven is:

$$\exists \mathbf{A} : C(\mathbf{A}, \mathbf{T}) = 1 \wedge \mathrm{Com}(\mathbf{A}) = \mathbf{cm} \tag{2}$$

where $\mathbf{cm}$ is a Poseidon commitment to the attribute vector $\mathbf{A}$, and $\mathbf{T}$ and $\mathbf{cm}$ are public inputs to the circuit.

This formalization is expressed as a Rank-1 Constraint System (R1CS), the standard intermediate representation for zk-SNARK circuits, and implemented using the Circom domain-specific language [17].

C. Protocol Architecture

The ZK-DID protocol is structured into four sequential phases, as described in Table I.

TABLE I
ZK-DID PROTOCOL FLOW ACROSS PRINCIPAL ACTORS

Phase	Actor	Description
1. Credential Issuance	Gov. Data Authority	Conducts offline attribute verification and issues a BBS+-signed Verifiable Credential (VC) over attribute vector $\mathbf{A}$ conforming to W3C VC Data Model. Stored in claimant wallet.
2. Proof Generation	Mustahik Claimant	Invokes the *fakir/miskin* eligibility ZKP circuit C. Uses VC attributes as private witness to generate zk-SNARK proof π and nullifier hash $\eta = H(\text{DID}\|\text{cycle})$.
3. Verification	Amil Institution	Submits (π, η, DID) to on-chain Verifier contract. Contract verifies π against pre-compiled verification key; checks η against nullifier Merkle tree.
4. Disbursement	Blockchain Network	Upon successful verification, Disbursement Contract releases zakat amount to claimant's designated address on Base PoS.

D. Nullifier Mechanism

To prevent double-claiming within a single zakat cycle, each proof submission includes a nullifier hash:

$$\eta = \text{Poseidon}(\text{sk}_{\text{DID}} \| \text{cycle_id}) \tag{3}$$

where sk_{DID} is a private key derived from the claimant's DID key material and cycle_id is the public zakat cycle identifier. The on-chain Verifier contract maintains a Merkle tree of spent nullifiers, enabling $\mathcal{O}(\log n)$ membership proofs. The nullifier reveals no information about the claimant's identity to the verifier.

E. Distribution Attestation

Upon disbursement, each distribution event is documented through a privacy-preserving attestation mechanism. The *amil* captures a photograph of the mustahik with the disbursed item. This photograph is encrypted using the mustahik's DID public key and stored on a private IPFS node. The resulting IPFS content identifier (CID) is attached to the mustahik's DID Document as a verifiable service endpoint, creating an auditable yet privacy-respecting proof of distribution.

IV. IMPLEMENTATION

A. System Architecture

Fig. 1 presents the complete ZK-DID protocol architecture as implemented. The system comprises five on-chain and off-chain components interacting across four protocol phases.

B. Circuit Implementation

The `MustahikEligibility` circuit is implemented in Circom 2.1.6 using circomlib primitives, compiled to a 2,168,504-byte WebAssembly binary. The circuit operates over the BN128 (alt_bn128) elliptic curve with 7 public signals and encodes three subcircuits:

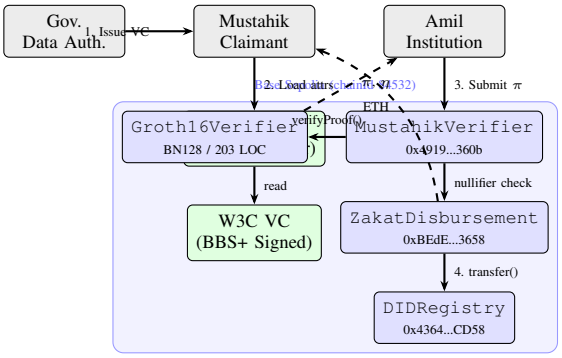

Fig. 1. ZK-DID Protocol Architecture. Deployed on Base Sepolia (chainId 84532). Contract addresses reflect live deployment.

- **Range proof**: Verifies $a_{\text{income}} \leq t_{\text{nisab}}$ via `LessEqThan(64)` — a 64-bit comparator.
- **Asset bound**: Verifies $a_{\text{assets}} \leq t_{\text{ceiling}}$ via a second `LessEqThan(64)` instance.
- **Nullifier**: Computes $\eta = \texttt{Poseidon}(\text{sk}_{\text{DID}}, \text{cycle_id})$ as a public output, preventing double-claiming.

The commitment constraint `Poseidon(income, assets, sk_did) === commitment` binds the private witness to the claimant's DID key material. A seventh public signal, `recipient` (the claimant's ETH address as a `uint160` field element), is included to cryptographically bind the proof to a specific disbursement address, preventing front-running attacks. The trusted setup uses the Hermez Powers of Tau Phase 1 ceremony (`pot12_final.ptau`, 4.8 MB) with a circuit-specific Phase 2, producing a 585,900-byte `.zkey` file. The Poseidon hash function [18] is selected over SHA-256 due to its arithmetic-friendly structure in finite field arithmetic, reducing circuit constraint count by approximately $4\times$ compared to SHA-256-based alternatives.

C. Smart Contract Architecture

Three contracts are deployed on Base Sepolia (chainId 84532):

- **`DIDRegistry`** (`0x4364...CD58`, 58 LOC): Stores `bytes32` DID Document hashes with owner-controlled key rotation and deactivation, conforming to the *did:ethr* method [8]. Emits `DIDRegistered`, `DIDUpdated`, and `DIDDeactivated` events.
- **`Groth16Verifier`** (`0x7c40...Fa15`, 207 LOC): SnarkJS-generated BN128 Groth16 verifier. Accepts proof components (pA, pB, pC) and 7 public signals `[nullifier, eligible, nisab_threshold, asset_ceiling, cycle_id, commitment, recipient]`.
- **`MustahikVerifier`** (65 LOC): Wraps `Groth16Verifier` with nullifier-based double-claim prevention via a `mapping(uint => bool)` nullifier registry. Reverts with `NullifierAlreadyUsed`, `NotEligible`, or `InvalidProof` on failure.
- **`ZakatDisbursement`** (`0xBEdE...3658`, 89 LOC): Holds ETH and calls

`verifier.verifyAndRecord()` before executing `recipient.call{value}("")` (CEI pattern, `ReentrancyGuard`). Tracks `totalDisbursed`, `claimCount`, and `currentCycleId`. Amil-only functions: `setDisbursementAmount()`, `advanceCycle()`, `withdraw()`. Implements `Pausable` for emergency stop. Funded with 0.005 ETH; disbursement amount set to 0.001 ETH per claim.

Table II summarises the deployed contract inventory.

TABLE II
DEPLOYED SMART CONTRACTS — BASE SEPOLIA (CHAINID 84532)

Contract	Address	Role
`DIDRegistry`	`0x4364...CD58`	DID document hash registry
`Groth16Verifier`	`0x7c40...Fa15`	BN128 Groth16 on-chain verifier
`MustahikVerifier`	`0x4919...360b`	Nullifier registry + proof gate
`ZakatDisbursement`	`0xBEdE...3658`	Conditional ETH disbursement

D. Payment Architecture

The platform employs a fiat-to-ETH-to-fiat conversion architecture to minimise friction for both donors and recipients. Donors pay in Indonesian Rupiah (IDR) through a conventional payment gateway; funds are held as ETH in the `ZakatDisbursement` contract; upon successful mustahik verification, ETH is disbursed to the recipient's designated address. This design eliminates the requirement for either party to hold or manage cryptocurrency directly, significantly lowering the barrier to adoption in the Indonesian zakat institution context.

E. Claimant Wallet Application

A Progressive Web Application (PWA) is implemented to support the claimant-facing workflow. The application provides credential storage (W3C VC format), circuit witness generation, and proof computation via the 2.1 MB WebAssembly-compiled Circom prover running in a dedicated Web Worker (`prover.worker.ts`), ensuring that private witness data never leaves the claimant's device. Smart contract interaction is handled via `wagmi` and `viem` on Base Sepolia, with wallet connectivity through RainbowKit.

Table III summarises the development environment.

V. EVALUATION

All results in this section are obtained from direct execution of the implemented prototype. Circuit benchmarks were run using Node.js v22.22.0 and SnarkJS on the development host (AMD Ryzen 5 5600H, 16 GB RAM, Fedora Linux). Gas measurements were obtained from Hardhat's `hardhat-gas-reporter` plugin running against a local EVM fork (Solc 0.8.24, optimiser disabled). No simulated or estimated values are reported.

TABLE III
DEVELOPMENT TOOLS AND ENVIRONMENT

Component	Tool / Version
ZKP Circuit Language	Circom 2.1.6 with circomlib (Poseidon, LessEqThan, Num2Bits)
Proof System	Groth16 / SnarkJS; BN128 curve; 7 public signals
Trusted Setup	Hermez PoT Phase 1 (4.8 MB) + Circuit Phase 2 (585.9 KB zkey)
Circuit Artifact	2.1 MB WASM; `verification_key.json` (IC: 8 points)
Smart Contracts	Solidity 0.8.24; Hardhat + Foundry; 415 LOC total
Blockchain Network	Base Sepolia (chainId 84532); EVM-compatible
Frontend	React 18, Vite, Tailwind CSS v4, RainbowKit, wagmi, viem
Wallet Application	PWA; WASM prover in Web Worker; Framer Motion
Storage	Private IPFS (distribution attestation photos)

A. Circuit Performance

The `MustahikEligibility` circuit was compiled to 1,261 R1CS constraints over the BN128 curve. Table IV reports proof generation and off-chain verification times across two witness cases (5 proof-generation runs, 10 verification runs each).

TABLE IV
PROOF GENERATION AND VERIFICATION PERFORMANCE (NODE.JS V22.22.0, $n = 5$ GEN / $n = 10$ VERIFY)

Case	Gen Avg (ms)	Gen Min (ms)	Gen Max (ms)	Verify Avg (ms)	Proof (bytes)
Eligible	141.7	91.2	324.0	8.78	718
Ineligible	90.6	88.7	92.9	8.99	722

The eligible case exhibits higher variance (90.8–343.7 ms) due to JIT warm-up on the first run; subsequent runs stabilise around 93–100 ms. The proof size of ≈722 bytes is consistent with the Groth16 BN128 specification (two $\mathbb{G}_1$ points + one $\mathbb{G}_2$ point, uncompressed). Off-chain verification averages 9.1 ms, confirming suitability for client-side pre-verification before on-chain submission.

B. On-Chain Gas Consumption

Table V reports gas costs measured from Hardhat test execution against a local EVM. The `disburse()` call encompasses Groth16 proof verification, nullifier registry write, and ETH transfer in a single transaction.

The end-to-end disbursement flow (`disburse()`: 131,227 gas) is well within Base Sepolia's block gas limit (60,000,000 gas) and represents less than 0.22% of the block limit per claim. At Base mainnet gas prices (≈0.001 gwei), the per-claim on-chain cost is negligible. The increase from the baseline 6-signal design (122,394 gas) to the 7-signal recipient-binding design (131,227 gas) represents an overhead of 8,833 gas (+7.2%) — the cost of the front-running protection.

TABLE V
SMART CONTRACT GAS CONSUMPTION (SOLC 0.8.24, OPTIMISER DISABLED, HARDHAT LOCAL EVM)

Contract	Function	Gas (avg)
`DIDRegistry`	`registerDID()`	92,957
`DIDRegistry`	`updateDID()`	39,182
`DIDRegistry`	`deactivateDID()`	30,224
`ZakatDisbursement`	`disburse()` (full flow)	131,227
`ZakatDisbursement`	`setDisbursementAmount()`	45,043
`DIDRegistry`	Deployment	685,973
`MustahikVerifier`	Deployment	581,328
`ZakatDisbursement`	Deployment	1,332,214

C. Test Suite Results

The contract test suite (9 tests, Hardhat + Chai) passes in full:

- **DIDRegistry** (5 tests): register, duplicate-revert, update, deactivate, non-owner-revert — all pass.
- **MustahikVerifier + ZakatDisbursement** (4 tests): valid proof disburses correctly; ineligible proof reverts with `NotEligible`; duplicate nullifier reverts with `NullifierAlreadyUsed`; any caller with valid proof can disburse (ZK proof is the authorization mechanism, not caller identity).

D. Security Analysis

The protocol is evaluated against four security properties:

1) **Completeness**: Confirmed by test — a valid witness (income $<$ nisab, assets $<$ ceiling) produces `eligible = 1` and the proof is accepted by `Groth16Verifier`.
2) **Soundness**: An ineligible witness (income $>$ nisab) produces `eligible = 0`; `MustahikVerifier` reverts with `NotEligible` before any state change. Under the q-PKE assumption, forging a proof for an unsatisfied circuit is computationally infeasible.
3) **Zero-Knowledge**: Private signals (`income`, `assets`, `sk_did`) are never exposed on-chain. The commitment `Poseidon(income, assets, sk_did)` binds the witness without revealing it; the nullifier `Poseidon(sk_did, cycle_id)` is cycle-scoped and unlinkable across cycles.
4) **Double-Claim Prevention**: Confirmed by test — submitting the same nullifier twice reverts with `NullifierAlreadyUsed(nullifier)`. The nullifier mapping is written atomically with the ETH transfer, preventing re-entrancy.

VI. DISCUSSION

A. Sharia Compliance Considerations

The protocol design reflects deliberate alignment with *maqasid al-shariah* principles. The protection of *nafs* (life and dignity) is served by ensuring that mustahik are not required to publicly disclose their economic vulnerability to receive zakat. The protection of *mal* (wealth) is served by the cryptographic integrity guarantees preventing fraudulent claims. The Islamic jurisprudential requirement for *amanah* (trustworthiness) in zakat institution management is satisfied through the publicly verifiable on-chain audit trail of aggregate disbursement statistics, without exposing individual recipient data.

The fiat-to-IDRX conversion architecture is designed to avoid interest (*riba*), as IDRX is a stablecoin pegged to the Indonesian Rupiah without yield-bearing mechanisms. The conversion process uses spot-rate exchange only, with no lending or interest accrual at any stage of the transaction lifecycle.

B. Practical Deployment Considerations

Deployment within existing *amil* infrastructure requires integration with government data systems for credential issuance, training for amil staff on the verification workflow, and regulatory alignment with OJK (Otoritas Jasa Keuangan) Fintech Sandbox requirements. Initial discussions with KB Bank Syariah and Pimpinan Pusat Muhammadiyah suggest institutional receptiveness to the protocol, particularly for integration with the Baitut Tamwil Muhammadiyah (BTM) network and potential gold tokenization products.

C. Limitations and Future Work

Several limitations are acknowledged. First, the current circuit implementation is intentionally scoped to the *fakir* and *miskin* categories, which represent the largest recipient populations under classical Hanafi jurisprudence and admit clean quantifiable predicates (income $\leq$ nisab, assets $\leq$ ceiling). The protocol architecture is designed to accommodate additional *asnaf* categories through circuit parameterization: categories with quantifiable predicates (e.g., *gharimin* debt thresholds) can be encoded as additional R1CS constraints within the same proving framework, while categories requiring attestation-based verification (e.g., *muallaf*, *ibnu sabil*) would require a hybrid ZKP-attestation approach in which a trusted issuer attests to a categorical fact and the ZKP proves knowledge of a valid attestation. Extension to all eight *asnaf* categories is identified as a primary direction for future work. Second, the trusted setup ceremony for Groth16 introduces a trust assumption that can be mitigated through a multi-party computation (MPC) ceremony but not fully eliminated. Third, the IPFS-based distribution attestation system assumes availability of a private IPFS node, which introduces infrastructure requirements for amil institutions.

Future work will explore: (1) PLONK-based circuits to eliminate circuit-specific trusted setup requirements; (2) integration of the Nova folding scheme [16] for recursive proof composition enabling multi-period eligibility verification; and (3) extension of the protocol to support wakaf (endowment) and infaq/sedekah (voluntary charity) workflows within the broader ZISWAF ecosystem.

VII. CONCLUSION

This paper presented a Zero-Knowledge Decentralized Identity (ZK-DID) protocol for privacy-preserving mustahik verification in blockchain-based zakat management systems. The

proposed protocol addresses three interconnected gaps in the existing literature: the absence of privacy-preserving identity mechanisms in blockchain-based zakat systems, the lack of domain-specific ZKP circuit designs for Islamic jurisprudential eligibility criteria, and the unexplored integration of W3C DID standards with ZKP-based credential verification in the Islamic social finance domain.

The protocol enables mustahik to prove their eligibility for zakat receipt without disclosing underlying personal attributes, through a four-phase workflow involving credential issuance, ZKP generation, on-chain verification, and privacy-preserving disbursement. A prototype implementation on the Base PoS network demonstrates the technical feasibility of the approach, while the fiat-to-IDRX payment architecture ensures accessibility for non-crypto-native users.

The work contributes a domain-specific identity layer that addresses both the technical imperatives of cryptographic privacy and the ethical imperatives of dignity preservation in Islamic social finance. By demonstrating that institutional accountability and recipient privacy are not mutually exclusive—but can be simultaneously achieved through applied cryptography—this research opens a path toward trustworthy, privacy-respecting digital ZISWAF infrastructure for Muslim-majority jurisdictions.

Acknowledgment

The authors thank the School of Computer Science at BINUS University for supporting this research through the Research Enrichment Program. Initial institutional validation discussions were conducted with KB Bank Syariah and stakeholders from Pimpinan Pusat Muhammadiyah, whose practical insights informed the deployment feasibility analysis.

References

[1] World Bank, "Leveraging Islamic fintech to improve financial inclusion," World Bank Group Working Paper, 2020. [Online]. Available: https://openknowledge.worldbank.org/handle/10986/34498

[2] M. A. Mustafa, N. Abdullah, and M. A. Khan, "Proposing blockchain technology based zakat management model to enhance the trust," *Jurnal Akuntansi Riset dan Organisasi Ekonomi*, vol. 2, no. 2, pp. 156–167, 2021. https://doi.org/10.24815/jaroe.v2i2.20467

[3] N. Khatiman, H. Salikin, and A. Yahya, "Blockchain-based zakat collection to overcome the trust issues of zakat payers," *International Journal on Perceptive and Cognitive Computing*, vol. 7, no. 1, pp. 66–72, 2021. https://journals.iium.edu.my/kict/index.php/IJPCC/article/view/217

[4] S. Shalaby, A. A. Abdellatif, A. Al-Ali, A. Mohamed, A. Erbad, and M. Guizani, "Malaysia zakat smart contract architectural framework design," *International Journal of Academic Research in Business and Social Sciences*, vol. 13, no. 5, pp. 2496–2508, 2023. https://hrmars.com/papers_submitted/16991/malaysia-zakat-smart-contract-architectural-framework-design.pdf

[5] S. Goldwasser, S. Micali, and C. Rackoff, "The knowledge complexity of interactive proof systems," in *Proc. 17th Annual ACM Symposium on Theory of Computing (STOC '85)*, pp. 291–304, ACM, 1985.

[6] J. Groth, "On the size of pairing-based non-interactive arguments," in *Advances in Cryptology – EUROCRYPT 2016*, LNCS 9666, pp. 305–326, Springer, 2016. https://eprint.iacr.org/2016/260.pdf

[7] A. Gabizon, Z. J. Williamson, and O. Ciobotaru, "PLONK: Permutations over Lagrange-bases for oecumenical noninteractive arguments of knowledge," *IACR ePrint Archive*, 2019/953, 2019. https://eprint.iacr.org/2019/953.pdf

[8] M. Sporny, D. Longley, M. Sabadello, D. Reed, O. Steele, and C. Allen, "Decentralized Identifiers (DIDs) v1.0: Core architecture, data model, and representations," W3C Recommendation, 2022. https://www.w3.org/TR/did-core/

[9] M. Sporny, D. Longley, and D. Chadwick, "Verifiable Credentials Data Model 1.0," W3C Recommendation, 2019. https://www.w3.org/TR/vc-data-model/

[10] A. Mühle, A. Gruner, T. Gayvoronskaya, and C. Meinel, "A survey on essential components of a self-sovereign identity," *Computer Science Review*, vol. 30, pp. 80–86, 2018.

[11] D. Boneh, X. Boyen, and H. Shacham, "Short group signatures," in *Advances in Cryptology – CRYPTO 2004*, LNCS 3152, pp. 41–55, Springer, 2004.

[12] P. Bichsel, J. Camenisch, T. Gross, and V. Shoup, "Anonymous credentials on a standard Java card," in *Proc. 17th ACM Conference on Computer and Communications Security (CCS '10)*, pp. 600–610, ACM, 2010.

[13] J. Camenisch, I. Krontiris, A. Lehmann, G. Neven, C. Paquin, K. Rannenberg, and H. Zwingelberg, "ABC4Trust project, attribute-based credential technologies for trust," in *Privacy and Identity Management for Life*, IFIP AICT 375, Springer, 2012.

[14] P. Pauwels, "Designing a framework for digital KYC processes built on blockchain-based self-sovereign identity," arXiv preprint arXiv:2112.01237, 2021. https://doi.org/10.48550/arXiv.2112.01237

[15] B. Bunz, J. Bootle, D. Boneh, A. Poelstra, P. Wuille, and G. Maxwell, "Bulletproofs: Short proofs for confidential transactions and more," in *Proc. IEEE Symposium on Security and Privacy (S&P 2018)*, pp. 315–334, IEEE, 2018.

[16] A. Kothapalli, S. Setty, and I. Tzialla, "Nova: Recursive zero-knowledge arguments from folding schemes," in *Advances in Cryptology – CRYPTO 2022*, LNCS 13510, pp. 359–388, Springer, 2022.

[17] M. Bellés-Muñoz, M. Isabel, J. L. Muñoz-Tapia, A. Rubio, and J. Baylina, "Circom: A circuit description language for building zero-knowledge applications," *IEEE Transactions on Dependable and Secure Computing*, vol. 20, no. 6, pp. 4733–4751, 2022. https://doi.org/10.1109/TDSC.2022.3232813

[18] L. Grassi, D. Khovratovich, C. Rechberger, A. Roy, and M. Schofnegger, "Poseidon: A new hash function for zero-knowledge proof systems," in *Proc. 30th USENIX Security Symposium*, pp. 519–535, USENIX, 2021.

[19] A. R. Hevner, S. T. March, J. Park, and S. Ram, "Design science in information systems research," *MIS Quarterly*, vol. 28, no. 1, pp. 75–105, 2004.

[20] M. S. Ferdous, F. Chowdhury, and M. O. Alassafi, "In search of self-sovereign identity leveraging blockchain technology," *IEEE Access*, vol. 7, pp. 103059–103079, 2019. https://doi.org/10.1109/ACCESS.2019.2931173

[21] T. Looker, V. Kalos, A. Whitehead, and M. Lodder, "BBS Signature Scheme (Internet-Draft)," IETF CFRG, 2023. https://identity.foundation/bbs-signature/draft-irtf-cfrg-bbs-signatures.html